Latent-Function Video Diffusion: Generate Once, Query Videos at Any Continuous Control

Anonymous CVPR submission

Paper ID *****

Abstract

*Video diffusion models normally run one denoising trajectory for every requested control value. We introduce **Latent-Function Video Diffusion (LFVD)**, which instead samples one family-valued latent object and queries complete videos from it at arbitrary continuous controls. A learned family encoder and lightweight query decoder are trained in frozen video-VAE space; a pretrained Video DiT then learns the family-code distribution with standard flow matching. On five object-disjoint Lighting folds, LFVD uses one trajectory for nine queries and attains LPIPS .3216 versus .3630 for a strengthened shared-noise Pointwise model with explicit control tokens, while their control correlations are statistically tied (.933 versus .930). A high-exposure audit shows that Pointwise can approach LFVD's per-query quality with nine times as many updates, so we do not attribute the matched-update fidelity gap solely to the representation. Instead, LFVD's distinctive evidence is functional: its generated query path matches the measured target path substantially better (adjacent-step LPIPS error .030 versus .087), and its learned reader improves LPIPS from .407 to .327 over linear interpolation of the same sampled family code. With three observed Lighting controls, a nine-times-update Pointwise stress test and LFVD trade LPIPS against PSNR and ordering at six measured unseen controls. On MEAD, LFVD remains comparable in fidelity but not superior in ordering. LFVD becomes faster than memory-feasible batched Pointwise at approximately seven Lighting queries. These results support family-valued diffusion as an amortized interface for coherent continuous video queries rather than an unconditional quality advantage.*

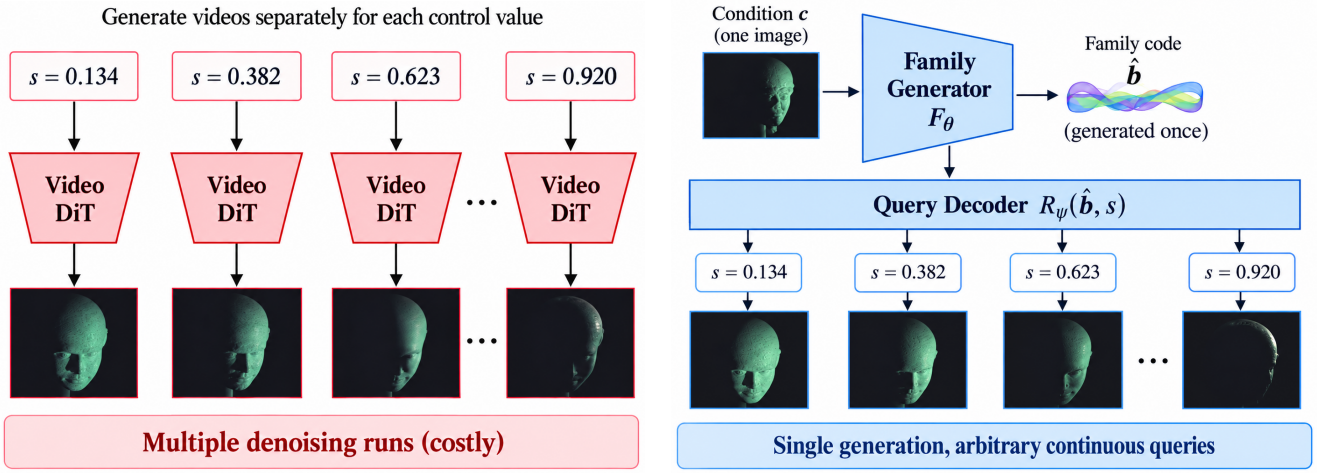
1. Introduction

Video diffusion models have made substantial progress in generating videos from text and image conditions [2, 11, 12, 24]. They also support continuous control over video attributes such as motion intensity and camera speed. In interactive workflows, users often explore several control values before selecting an output. These requests may depict the same scene and subject while changing one attribute. The resulting videos share much of their visual content, yet generating each video separately repeats expensive computation for content that is largely unchanged. This motivates reusing information across control values.

Conventional video diffusion models generate each requested video separately. Given a control value s, the model runs a complete denoising process to produce the corresponding video. We refer to this approach as *pointwise generation*. Using the same initial noise can improve consistency across videos, but the model still performs a separate denoising process for each control value. Another practical approach generates videos at selected endpoints and interpolates their latents to obtain intermediate videos. This reduces repeated sampling, while the intermediate outputs follow a fixed interpolation rule.

We introduce **Latent-Function Video Diffusion (LFVD)**, a framework that generates videos across a continuous control range using a single diffusion process. LFVD generates a shared latent representation of videos at different control values. A lightweight query decoder takes this representation and a requested value $s \in [0, 1]$ to produce a complete video latent, which a frozen video VAE converts into video frames. The same representation supports multiple queries without additional denoising. Figure 1 illustrates this process.

LFVD learns the shared representation and its generation model in two stages. First, a frozen video VAE encodes videos observed at different control values. A family encoder combines their video latents into one representation, and a query decoder learns to reconstruct each video latent from the representation and its control value. We train these components using latent reconstruction loss. Next, we freeze the family encoder and query decoder and train a pretrained Video DiT to generate shared representations using standard flow matching. During inference, the Video DiT samples one representation from the input condition. The query decoder then produces videos at requested con-



(a) Pointwise generation: one diffusion process per control value.

(b) LFVD: one diffusion process, multiple video queries.

Figure 1. **Generate once, query many videos.** Pointwise generation runs a separate denoising process for each requested control value s. LFVD generates a shared latent representation once and uses a lightweight query decoder to produce videos at different control values. The thumbnails show held-out OpenIllumination frames. Multi-query costs are measured in Sec. 4.6.

trol values without requiring the original videos or further Video DiT evaluations.

We compare three ways to construct the shared representation. LFVD-Identity directly concatenates the observed video latents. LFVD-Mix learns to combine information across latent time and control positions, while LFVD-Compact uses a shorter representation. All three use the same query decoder and diffusion training procedure. Their comparison helps identify how representation design affects generation quality and computational cost.

The shared representation contains more tokens than a single video latent, so its diffusion process has a higher initial cost. LFVD can offset this cost when multiple videos are requested. We measure the trade-off using synchronized inference time, analytical transformer FLOPs, and representation-size comparisons.

We evaluate LFVD on lighting and facial-expression control. OpenIllumination provides measured views of the same objects under various lights, which we arrange along a fixed camera path to form videos. MEAD provides facial videos at three ordered expression-intensity levels. We evaluate on unseen objects and actors and compare with pointwise generation and interpolation between endpoint video latents.

We strengthen Pointwise with an explicit polynomial–Fourier control token selected on validation data. At the matched update budget, LFVD improves Lighting LPIPS from .3630 to .3216 and PSNR from 19.78 to 21.14 dB, while control correlation is tied (.930 versus .933). A separate nine-times-update audit narrows or reverses the per-query gap depending on the metric, showing that the matched-update advantage partly reflects family-level supervision density. The family representation nevertheless provides a distinct functional benefit. Relative to real measured query paths, LFVD reduces adjacent-step LPIPS error from .0867 to .0303 compared with shared-noise Pointwise. On the same sampled family codes, the learned query decoder lowers LPIPS from .4074 to .3269 compared with adjacent linear interpolation. With three observed Lighting controls, LFVD and an aggressively trained Pointwise model trade perceptual fidelity against PSNR and ordering at six measured unseen controls. In facial-expression control, LFVD maintains comparable fidelity, although endpoint interpolation achieves stronger ordering. Synchronized timing shows that LFVD becomes faster than batched Pointwise after approximately seven Lighting queries. These results position LFVD as an amortized, queryable video-family generator rather than a universal per-query quality improvement.

Contributions.

- We introduce LFVD, a framework that generates a shared latent representation with one diffusion process and uses a lightweight query decoder to produce complete videos at continuous control values.
- We develop a two-stage training procedure that learns a family encoder and query decoder, followed by training a Video DiT using standard flow matching.
- We show that sampled LFVD codes produce query paths closer to measured continuous families than Pointwise or latent interpolation, and that the learned reader materially outperforms linear interpolation of the same code.
- We evaluate held-out Lighting and facial-expression families, sparse measured controls, exposure, encoder design,

and synchronized multi-query cost, including audits that delimit rather than hide the source of the observed gains.

2. Related Work

2.1. Video Diffusion and Continuous Control

Video diffusion and continuous-control methods generate videos from visual conditions and allow users to adjust specific attributes. Early models developed spatiotemporal denoising and large-scale video synthesis [11, 12, 24]. Latent video diffusion operates on compressed representations, while Lumiere and Wan advance spatiotemporal modeling and pretrained video generation [1, 2, 27]. Motion-control methods guide generation using trajectories, motion fields, and regional constraints [9, 22, 29, 32]. Go-with-the-Flow uses warped noise, and TokenMotion separates human and camera motion [3, 13]. MotionStone and Mojito control motion intensity, while Text Slider, FreeSliders, and TokenDial provide fine-grained attribute control [4, 7, 10, 18, 21].

2.2. Joint and Function-based Generation

Joint and function-based generation methods model related outputs and signals across multiple coordinates. Multi-view diffusion methods generate consistent images from different viewpoints: MVDream jointly models camera views, and SyncDreamer uses relationships among views conditioned on a reference image [17, 23]. Functional generative models represent signals over continuous input domains. Neural Diffusion Processes model distributions over functions [5]. Score-Based Diffusion Models in Function Space and Continuous-Time Functional Diffusion Processes extend diffusion to function-valued data [8, 14]. Functional Diffusion studies continuous signal generation, while HyperDiffusion generates neural-field parameters with diffusion [6, 30].

2.3. Efficient Diffusion Sampling

Efficient diffusion sampling methods reduce the computational cost of generating samples. DDIM introduces an alternative sampling procedure, while progressive distillation trains models to generate samples in fewer steps [20, 25]. Consistency Models and Latent Consistency Models learn consistency across noise levels to enable fast sampling [19, 26]. Other practical strategies for generating related outputs include reusing the same initial noise to improve consistency and interpolating the latents of generated endpoints to produce intermediate results. These approaches support efficient generation through reduced sampling steps, shared randomness, or interpolation.

3. Latent-Function Video Diffusion

LFVD generates a shared latent representation that supports videos at continuous control values. We first train a family encoder and a query decoder using videos observed at different controls. We then train a Video DiT to generate the shared representation. At inference, the same generated representation supports multiple video queries. Figure 2 illustrates the two training stages and inference process.

3.1. Conditional Videos and Latent Functions

Let c denote the shared input condition, including a reference image and text, and let $s \in \mathcal{S} = [0, 1]$ be a continuous control value. We write x_s for the video at control value s. A frozen video VAE consists of an encoder $\mathcal{E}_{\mathrm{VAE}}$ and a decoder $\mathcal{D}_{\mathrm{VAE}}$. We denote its video latent by $z_s = \mathcal{E}_{\mathrm{VAE}}(x_s)$. Let C, T_z, H, and W denote the channel, temporal, height, and width dimensions of the *VAE latent*, respectively. Thus, $z_s \in \mathbb{R}^{C\times T_z\times H\times W}$; H and W refer to latent spatial dimensions, not the resolution of the original video.

For pointwise generation, let $G_{\theta_{\mathrm{PW}}}$ be a conditional diffusion sampler with parameters θ_{PW}. We use ε for its initial Gaussian noise, $\hat{z}_s$ for the generated video latent, and $\hat{x}_s$ for the decoded video. Pointwise generation is

$$\hat{z}_s = G_{\theta_{\mathrm{PW}}}(c, s, \varepsilon), \qquad \hat{x}_s = \mathcal{D}_{\mathrm{VAE}}(\hat{z}_s). \tag{1}$$

The sampler performs a complete denoising process for each requested control value. LFVD instead uses a shared latent representation $\mathbf{b}$ and a query decoder R_ψ, parameterized by ψ, to reconstruct an individual video latent. Let $\mathcal{Z}_{\mathbf{b}}$ denote the resulting mapping from control values to video latents:

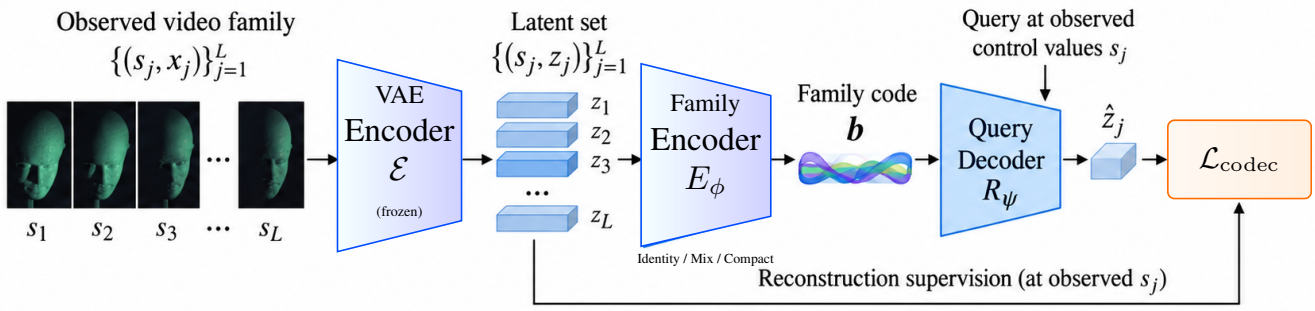
$$\begin{aligned} \mathcal{Z}_{\mathbf{b}} &: \mathcal{S} \to \mathbb{R}^{C\times T_z\times H\times W}, \\ \mathcal{Z}_{\mathbf{b}}(s) &= R_\psi(\mathbf{b}, s). \end{aligned} \tag{2}$$

A single representation therefore provides a video latent for each requested control value.

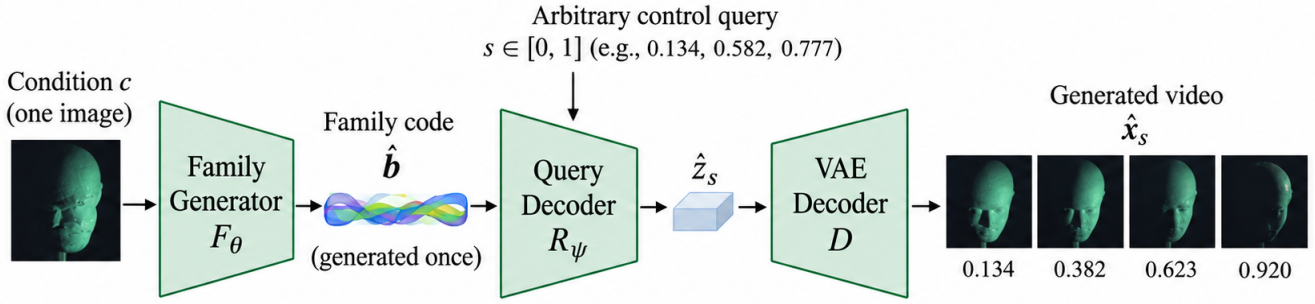
3.2. Learning a Family Representation

For each shared condition c, let L denote the number of observed videos and let $\ell \in \{1, \dots, L\}$ index their control values s_ℓ. We write $z_\ell = \mathcal{E}_{\mathrm{VAE}}(x_{s_\ell})$ for the corresponding latents and define the training family as $\mathcal{F}_c = \{(s_\ell, x_{s_\ell})\}_{\ell=1}^{L}$. We divide each video latent along the temporal dimension into a shared reference frame $z_{\mathrm{shared}} \in \mathbb{R}^{C\times 1\times H\times W}$ and the remaining $T_z - 1$ frames $z_{\ell,\mathrm{rest}}$, giving $z_\ell = [z_{\mathrm{shared}}, z_{\ell,\mathrm{rest}}]$. The total number of non-reference latent frames across the observed videos is $J = L(T_z - 1)$.

Let E_ϕ denote the family encoder, with parameters ϕ when learned. It combines the non-reference video latents into an encoded representation $\mathbf{b}_{0,\mathrm{rest}}$ containing J' latent frames. Our encoder preserves the VAE latent channel and spatial dimensions C, H, and W, and transforms only the latent-time dimension. Therefore, $\mathbf{b}_{0,\mathrm{rest}} \in \mathbb{R}^{C\times J'\times H\times W}$.



(a) Learning the family encoder and query decoder from observed videos.



(b) Generating one shared representation and querying multiple videos.

Figure 2. **LFVD training and inference.** (a) A frozen video VAE encodes the observed videos. The family encoder and query decoder learn a shared latent representation through reconstruction. We then freeze these components and train a Video DiT with flow matching. (b) At inference, the Video DiT samples the non-reference portion of one shared representation, while the reference latent remains fixed. The query decoder and frozen VAE produce videos at requested control values without further denoising.

We concatenate this encoded representation with the unchanged reference frame to obtain the complete family code $\mathbf{b}_0$:

$$\begin{aligned} \mathbf{b}_{0,\text{rest}} &= E_\phi\big(\{(s_\ell, z_{\ell,\text{rest}})\}_{\ell=1}^{L}\big), \\ \mathbf{b}_0 &= [z_{\text{shared}}, \mathbf{b}_{0,\text{rest}}]. \end{aligned} \tag{3}$$

The family encoder can be fixed or learned.

Family encoder variants. We implement the family encoder E_ϕ in Eq. 3 using a temporal projection. Let $\text{Emb}_\phi(s) \in \mathbb{R}^C$ denote an optional control embedding, λ its scale, and $P_\phi \in \mathbb{R}^{J' \times J}$ a temporal projection. The embedding is broadcast over time and space, and the video latents are concatenated in control-value order:

$$\mathbf{b}_{0,\text{rest}} = P_\phi \,\text{Concat}_{\ell,\text{time}}\left(z_{\ell,\text{rest}} + \lambda\,\text{Emb}_\phi(s_\ell)\right). \tag{4}$$

The three encoder variants are defined by

$$\begin{aligned} \text{Identity:} \quad & J' = J,\ P_\phi = I,\ \lambda = 0, \\ \text{Mix:} \quad & J' = J,\ P_\phi \in \mathbb{R}^{J \times J}, \\ \text{Compact:} \quad & J' < J,\ P_\phi \in \mathbb{R}^{J' \times J}. \end{aligned}$$

Here, I is the identity matrix. Identity directly concatenates the observed latents, Mix learns temporal mixing and control embeddings, and Compact reduces the representation length.

Continuous query decoder. Given the family code $\mathbf{b}_0$ and a control value s, the query decoder R_ψ uses an s-conditioned neural network to generate a temporal projection $A_\psi(s)$ and a bias $\beta_\psi(s)$. Their dimensions and the decoding process are

$$\begin{aligned} A_\psi(s) &\in \mathbb{R}^{(T_z-1)\times J'}, \\ \beta_\psi(s) &\in \mathbb{R}^{C\times(T_z-1)}, \\ \hat{z}_{s,\text{rest}} &= A_\psi(s)\mathbf{b}_{0,\text{rest}} + \beta_\psi(s), \\ R_\psi(\mathbf{b}_0, s) &= [z_{\text{shared}}, \hat{z}_{s,\text{rest}}]. \end{aligned} \tag{5}$$

Representation capacity. Identity and Mix both use $J' = J$. On a fixed grid of observed control values, the query decoder can absorb an invertible temporal projection and the control-embedding offsets. These variants therefore have the same nominal reconstruction capacity when the Mix projection is invertible, although they can produce different code distributions for diffusion training. Compact reduces J' and introduces a compression–fidelity trade-off. We compare these effects in Sec. 4.5.

3.3. Diffusion over the Family Code

After codec training, we freeze E_ϕ and R_ψ and encode each training family into a clean code $\mathbf{b}_0$. Let θ denote

the trainable Video DiT parameters. The model learns the conditional distribution of family codes using standard flow matching [15].

Let $t \in [0,1]$ denote the normalized flow-matching noise level, and let ε be a Gaussian-noise draw with the same shape as $\mathbf{b}_{0,\text{rest}}$. We denote the noisy family code by $\mathbf{b}_t$ and the target velocity of its non-reference portion by $\mathbf{v}_{\text{target}}$. The reference frame z_{shared} remains fixed, while the non-reference portion is interpolated between its clean value and noise:

$$\begin{aligned} \mathbf{b}_t &= [z_{\text{shared}}, (1-t)\mathbf{b}_{0,\text{rest}} + t\varepsilon], \\ \mathbf{v}_{\text{target}} &= \varepsilon - \mathbf{b}_{0,\text{rest}}. \end{aligned} \tag{6}$$

Let $\mathbf{v}_{\theta,\text{rest}}$ denote the Video DiT's predicted velocity for the non-reference portion, and let $w(t)$ denote the loss weight defined by the pretrained scheduler. We write $\mathcal{L}_{\text{FM}}$ for the flow-matching loss and $\|\cdot\|_2^2$ for squared error over the predicted elements. The expectation covers training families, sampled noise levels, and Gaussian noise. We minimize

$$\mathcal{L}_{\text{FM}} = \mathbb{E}\left[w(t)\|\mathbf{v}_{\theta,\text{rest}}(\mathbf{b}_t, t, c) - \mathbf{v}_{\text{target}}\|_2^2\right]. \tag{7}$$

At inference, let F_θ denote the complete sampling process obtained by integrating the trained velocity predictor $\mathbf{v}_{\theta,\text{rest}}$. We use ε again for an independent Gaussian-noise draw, now with the shape of the family-code non-reference portion. The sampler keeps z_{shared} fixed and produces a family code $\hat{\mathbf{b}}$. Let $\hat{x}_s$ denote the video at the requested control s. Generation follows

$$\begin{aligned} \hat{\mathbf{b}} &= F_\theta(c, \varepsilon), \\ \hat{x}_s &= \mathcal{D}_{\text{VAE}}\Big(R_\psi(\hat{\mathbf{b}}, s)\Big). \end{aligned} \tag{8}$$

The requested control value enters only the query decoder. Once the family code has been sampled, each additional video requires a query-decoder and VAE-decoder evaluation.

3.4. Multi-Query Generation Cost

The computational cost of LFVD depends on the family-code length J' and the number of requested videos M. Let $\mathcal{C}_{\text{diff}}(n)$ denote the cost of diffusion sampling with n latent frames, keeping the backbone, spatial dimensions, and sampling steps fixed. Let $\mathcal{C}_D$ denote the cost of one VAE decoding and $\mathcal{C}_R(J')$ the cost of querying a family code of length $1 + J'$.

The query cost includes generating $A_\psi(s)$ and $\beta_\psi(s)$ and applying the temporal projection. Let $\mathcal{C}_{\text{param}}(J')$ denote the cost of generating these projection parameters. Counting each multiply–accumulate as two operations, the query cost is approximately

$$\mathcal{C}_R(J') \approx \mathcal{C}_{\text{param}}(J') + CHW(T_z - 1)(2J' + 1). \tag{9}$$

The total costs of pointwise generation and LFVD for M videos are

$$\begin{aligned} \mathcal{C}_{\text{PW}}(M) &= M\mathcal{C}_D + M\mathcal{C}_{\text{diff}}(T_z), \\ \mathcal{C}_{\text{LFVD}}(M) &= M\mathcal{C}_D + \mathcal{C}_{\text{diff}}(1 + J') \\ &\quad + M\mathcal{C}_R(J'). \end{aligned} \tag{10}$$

Both methods require M VAE decodings. The difference lies in the repeated diffusion cost of pointwise generation and the single diffusion cost followed by queries in LFVD.

Since the VAE costs cancel, LFVD becomes more efficient when

$$M > \frac{\mathcal{C}_{\text{diff}}(1 + J')}{\mathcal{C}_{\text{diff}}(T_z) - \mathcal{C}_R(J')}, \tag{11}$$

provided that $\mathcal{C}_{\text{diff}}(T_z) > \mathcal{C}_R(J')$. This condition gives the minimum query count needed to amortize the family-generation cost under the sequential cost model. We evaluate the actual crossover, including batched pointwise generation, in Sec. 4.6.

4. Experiments

We evaluate per-query fidelity, functional query-path consistency, and multi-query cost. We also test unseen controls, exposure, the query reader, encoder variants, and reconstruction error. Main results use held-out objects or actors, three seeds, and 50 denoising steps.

4.1. Experimental Setup

Datasets. *OpenIllumination* [16] captures real objects under one-light-at-a-time illumination. We fix a 17-frame camera path and vary nine measured light positions, normalized to $s \in [0,1]$ by angular arc distance. Five object-disjoint folds contain 48/16/16 training, validation, and test families each, totaling 80 held-out families.

MEAD [28] provides facial videos at three ordinal expression intensities (low, medium, high). We align recordings using dynamic time warping on mel-frequency cepstral coefficients and their temporal derivatives, then form 17-frame families. Four actor-disjoint folds use five training, one validation, and two test actors each, totaling 112 held-out families. MEAD contains identifiable faces and voices. We use this existing dataset under its research terms and collect no new participant data; no new IRB review or consent process was conducted.

Implementation. We use the Wan2.2-TI2V-5B Video DiT and frozen video VAE [27] with the official DiffSynth trainer. All videos have 17 RGB frames at 256×256 resolution. We train the family encoder and query decoder for 8,000 updates, freeze them, and train rank-64 LoRA adapters on Video DiT attention and MLP projections for 3,456 updates. Pointwise models use the

Table 1. **Main held-out results at the matched update budget.** LFVD denotes LFVD-Mix. Lighting Pointwise uses the validation-selected control token. All trajectories use 50 steps. Runs and calls are per complete measured-control family: nine outputs for Lighting and three for MEAD. Calls do not normalize for the longer LFVD token sequence and are not latency.

		Sampling work		Video fidelity			Control
Dataset	Method	Runs ↓	Calls ↓	LPIPS ↓	PSNR ↑	SSIM ↑	ρ ↑
OpenIllumination	Pointwise (independent)	9	450	0.3583	20.12	0.7393	0.8394
	Pointwise (shared noise)	9	450	0.3630	19.78	0.7318	0.9303
	Endpoint interpolation	2	100	0.4054	19.19	0.7126	**0.9944**
	LFVD (ours)	**1**	**50**	**0.3216**	**21.14**	**0.7526**	0.9326
MEAD	Pointwise (independent)	3	150	**0.1063**	**22.97**	**0.7643**	0.5774
	Pointwise (shared noise)	3	150	0.1157	22.54	0.7534	0.7426
	Endpoint interpolation	2	100	0.1152	22.91	0.7624	**0.8586**
	LFVD (ours)	**1**	**50**	0.1151	22.97	0.7597	0.7188

same backbone, optimizer, LoRA rank, number of updates, and splits. For Lighting, Pointwise receives an eight-dimensional polynomial–Fourier encoding of s through zero-initialized latent-bias and context-token projections. This design is selected on the L0 validation fold and then frozen across test folds. It adds 0.53M parameters to the 161.22M LoRA parameters. The LFVD codec has 10.49M parameters and receives s only through its query decoder. Richer polynomial–Fourier LFVD query features do not change sampled metrics. On MEAD, the token does not improve validation, so we retain the stronger scalar-conditioned Pointwise baseline. Sampling uses 50 steps with seeds $\{100, 200, 300\}$.

Baselines. *Pointwise independent* samples each control value with independent Gaussian noise. *Pointwise shared-noise* uses the same initial noise for all requested values and runs a complete denoising process for each value. *Endpoint interpolation* generates the two endpoint videos and linearly interpolates their VAE latents. Unless specified otherwise, LFVD denotes LFVD-Mix. We also evaluate LFVD-Identity and LFVD-Compact.

Metrics. We report AlexNet LPIPS [31], PSNR, and SSIM against measured targets. Control correlation ρ is the Spearman correlation between s and the target-projected response; constant outputs receive zero. For query paths, we compare generated and measured adjacent-query LPIPS, total path length, and second-order RGB variation. The latter two use absolute log ratios, so zero is ideal and neither collapse nor excess variation is rewarded. We use paired object- or actor-level cluster bootstrap intervals (95%). Full settings and per-sample results are in the supplement.

4.2. Main Results

On OpenIllumination (Table 1), LFVD improves LPIPS from .3630 to .3216 and PSNR from 19.78 to 21.14 dB relative to the strengthened shared-noise Pointwise. The paired differences favor LFVD by .0414 LPIPS (95% CI $[.0316, .0511]$) and 1.36 dB PSNR ($[1.14, 1.61]$). Control correlation is tied: .930 for Pointwise and .933 for LFVD, with a paired LFVD-minus-Pointwise interval of $[-.0208, .0243]$. LFVD uses one trajectory for nine values, compared with nine for Pointwise. Endpoint interpolation has the strongest rank ordering, but its fidelity is substantially lower.

On MEAD, LFVD matches shared-noise Pointwise fidelity (.1151 versus .1157 LPIPS) but has lower correlation (.719 versus .743). Endpoint interpolation reaches .859 correlation, despite LFVD's larger endpoint response. These results show different fidelity and ordering behavior across control domains. Figure 3 compares methods on two held-out Lighting objects and two held-out MEAD families across selected temporal frames. Endpoint latent interpolation can produce soft double facial contours, or ghosting, from blending endpoint states; this is most visible for M005 angry-015 in Figure 3(d). The selected examples expose method differences, while Table 1 remains the aggregate test. The supplement shows additional Lighting continua decoded from a single LFVD trajectory.

4.3. Exposure and Functional Query Audits

The matched-update comparison does not match the number of control latents processed per update: LFVD observes a complete family, whereas Pointwise observes one control. We therefore train dense Pointwise for nine times as many updates on the scene-disjoint L0 validation split. Pointwise reaches .3268 LPIPS and .998 correlation, while LFVD reaches .3365 and .973; LFVD remains slightly better in PSNR and SSIM. This single-fold stress test shows

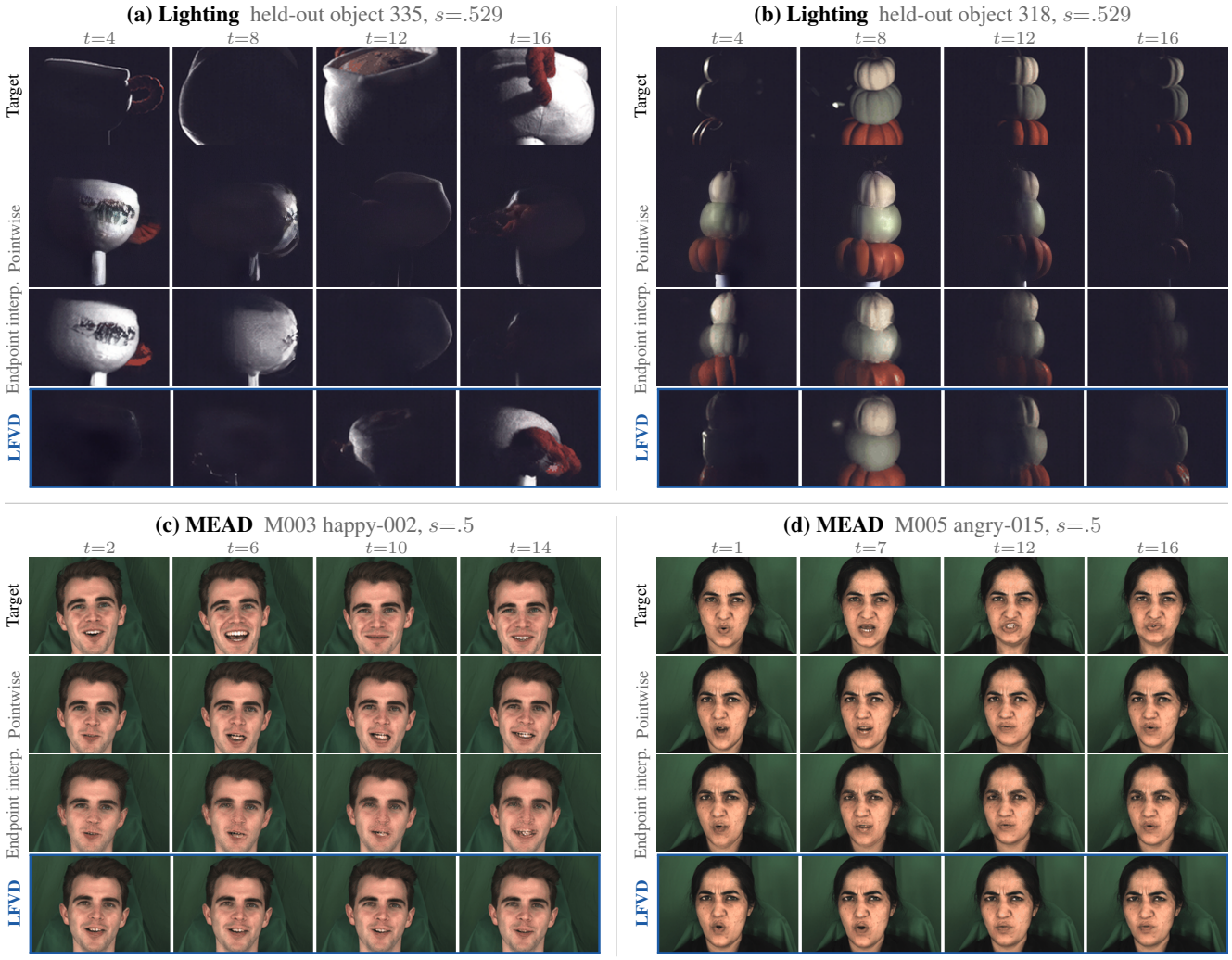


Figure 3. **Qualitative comparison on held-out Lighting and MEAD families.** Rows show the measured target, shared-noise Pointwise, endpoint latent interpolation, and LFVD (blue frame); columns show temporal frames. Lighting Pointwise uses the final control-token model. The top row contains two Lighting objects and the bottom row two MEAD expressions. Endpoint interpolation can leave soft double contours in (d), whereas LFVD produces a coherent facial trajectory. All panels use the locked 50-step test protocol without sharpening, alignment, or color correction.

that the main fidelity gap is not evidence of an unconditional representational advantage. It does not alter LFVD's one-trajectory inference interface; the complete table is in the supplement.

We next measure whether one sampled code represents the *path* traced by the real video family. Table 2 reports target-relative errors over nine Lighting queries. LFVD reduces adjacent-step LPIPS error from .0867 to .0303 against shared-noise Pointwise. Its path-length and second-order variation errors are also lowest. Paired object-bootstrap intervals for all three LFVD improvements exclude zero; the supplement provides them. This metric is target-relative, so a constant or overly smooth output is penalized rather than rewarded.

Table 2. **Target-relative query-path consistency on Lighting.** Errors compare each generated nine-query path with its measured target path; lower is better. Results use 80 held-out families and seed 100.

Method	Adj. LPIPS MAE	Path log err.	Jerk log err.
Pointwise independent	.1190	.5100	.4712
Pointwise shared	.0867	.5866	.5176
Endpoint interpolation	.1439	1.7460	2.1358
LFVD	**.0303**	**.1341**	**.1588**

Finally, we hold the sampled family code fixed and replace R_ψ with piecewise linear interpolation between adjacent packed latent segments. The learned reader lowers LPIPS by .0805 ($[.0488, .1132]$) and improves PSNR by

Table 3. **Sparse-control stress test.** Training uses controls $\{0, 4, 8\}$; evaluation uses all nine measured positions. Pointwise receives $9\times$ the updates. Means cover five folds, three seeds, and 50 steps.

	Obs.	Unseen controls		Control
Method	LPIPS↓	LPIPS↓	PSNR↑	ρ ↑
Pointwise token	**.2895**	**.3322**	20.58	.9691
LFVD-Mix	.2993	.3410	**21.22**	**.9906**

1.10 dB ($[.11, 2.13]$), while preserving essentially identical ordering. The gain therefore comes from learned continuous querying, not from sampling a different family code; the complete ablation is in the supplement.

4.4. Generalization to Unseen Controls

We train both methods using only OpenIllumination indices $\{0, 4, 8\}$ and evaluate all nine measured positions. The six intermediate positions are excluded from training, and targets are recorded measurements. To avoid repeating the weak-baseline issue, Pointwise receives 31,104 updates, nine times the main budget, while LFVD uses the unchanged sparse checkpoint. The control-token architecture was fixed before these test runs.

The result is deliberately mixed. At unseen controls, Pointwise has lower LPIPS (.3322 versus .3410), while LFVD has higher PSNR (21.22 versus 20.58 dB); SSIM is tied. LFVD better preserves global ordering over all nine queries (.991 versus .969). Paired object-bootstrap intervals confirm the LPIPS difference in favor of Pointwise ($[.0041, .0136]$ for LFVD minus Pointwise), the PSNR difference in favor of LFVD ($[.48, .79]$ dB), and the ordering difference in favor of LFVD ($[.011, .032]$). Thus LFVD demonstrates absolute measured-target generalization, but not uniform superiority over a high-exposure Pointwise model.

4.5. Family Encoder Ablation

Identity and Mix are statistically tied on full-grid Lighting (LPIPS difference, Identity minus Mix: $-.0014$, 95% CI $[-.0058, .0030]$) and sparse unseen controls ($+.0007$, $[-.0015, .0028]$). On MEAD, Identity has lower LPIPS by .0082 ($[.0002, .0204]$) and higher PSNR by .19 dB ($[.00, .46]$). Thus the family-valued formulation is robust to the encoder choice, while learned mixing is not uniformly beneficial. The supplement reports complete encoder metrics and paired intervals for both datasets.

4.6. Measured Multi-Query Cost

As defined in Eq. 10, Pointwise repeats diffusion for every requested video, whereas LFVD samples once and adds inexpensive queries. Against memory-feasible batch-3 Pointwise on one RTX 4090, the measured Lighting crossover is approximately seven queries: 15.40 versus 16.96 s at $M = 7$. At $M = 9, 17, 33$, LFVD is $1.23\times$, $1.97\times$, and $2.90\times$ faster. For MEAD's three controls, LFVD takes 5.74 s versus 6.60 s for batch-3 Pointwise. The supplementary FLOP audit gives diffusion-only crossovers of 6.82 queries for dense Lighting and 2.37 for MEAD. Sparse Lighting also uses a 13-frame code, versus 37 frames for the dense model, with a 2.37-query transformer-only crossover.

4.7. Ablations and Bottleneck Analysis

Exact-code LPIPS is .0642, close to the frozen-VAE floor of .0512, whereas sampling the same representation gives .3365. Family-code diffusion is thus the main bottleneck. LFVD-Compact reduces the analytical crossover from 6.82 to 5.09 queries but degrades LPIPS from .3300 to .4053. Complete rank, scaling, coordinate, and reconstruction ablations are in the supplement.

4.8. Limitations

LFVD's larger latent requires enough queries to amortize its cost. OpenIllumination is controlled multi-view data; MEAD has only three ordinal levels. Matched updates give LFVD denser supervision, and high-exposure Pointwise closes the per-query gap. Sparse controls show a metric trade-off, not uniform superiority. Encoder differences are small and dataset-dependent, Compact loses fidelity, and sampled family codes remain below exact-code reconstruction.

5. Conclusion

LFVD samples one reusable family object for post-hoc continuous video queries. At matched updates it improves Lighting fidelity over a control-aware Pointwise model, but a high-exposure audit shows that this is not an intrinsic quality guarantee. The stronger result is functional: LFVD follows measured query paths more faithfully, and its learned reader outperforms linear interpolation of the same sampled code. Sparse controls reveal a metric trade-off, MEAD preserves fidelity rather than superior ordering, and multi-query inference amortizes after enough requests. Identity matches Mix overall, locating the contribution in family-valued generation and querying.

LLM Usage Statement

OpenAI Codex was used as an engineering and writing assistant for research ideation, implementation and debugging of experiment code, experiment automation, preparation of figures and tables, numerical-consistency checks, and prose editing. The human authors defined the research questions and methodology, selected the data and evaluation protocols, inspected the generated outputs, verified the reported results and citations, and retain full responsibility for the submission.

References

[1] Omer Bar-Tal, Hila Chefer, Omer Tov, Charles Herrmann, Roni Paiss, Shiran Zada, Ariel Ephrat, Junhwa Hur, Guanghui Liu, Amit Raj, Yuanzhen Li, Michael Rubinstein, Tomer Michaeli, Oliver Wang, Deqing Sun, Tali Dekel, and Inbar Mosseri. Lumiere: A space-time diffusion model for video generation. *arXiv preprint arXiv:2401.12945*, 2024. 3

[2] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. In *CVPR*, pages 22563–22575, 2023. 1, 3

[3] Ryan Burgert, Yuancheng Xu, Wenqi Xian, Oliver Pilarski, Pascal Clausen, Mingming He, Li Ma, Yitong Deng, Lingxiao Li, Mohsen Mousavi, Michael Ryoo, Paul Debevec, and Ning Yu. Go-with-the-flow: Motion-controllable video diffusion models using real-time warped noise. In *CVPR*, pages 13–23, 2025. 3

[4] Pin-Yen Chiu, I-Sheng Fang, and Jun-Cheng Chen. Text slider: Efficient and plug-and-play continuous concept control for image/video synthesis via LoRA adapters. In *WACV*, 2026. 3

[5] Vincent Dutordoir, Alan Saul, Zoubin Ghahramani, and Fergus Simpson. Neural diffusion processes. In *ICML*, pages 8990–9012, 2023. 3

[6] Ziya Erkoç, Fangchang Ma, Qi Shan, Matthias Nießner, and Angela Dai. HyperDiffusion: Generating implicit neural fields with weight-space diffusion. In *ICCV*, 2023. 3

[7] Rotem Ezra, Hedi Zisling, Nimrod Berman, Ilan Naiman, Alexey Gorkor, Liran Nochumsohn, Eliya Nachmani, and Omri Azencot. FreeSliders: Training-free, modality-agnostic concept sliders for fine-grained diffusion control in images, audio, and video. *arXiv preprint arXiv:2511.00103*, 2025. 3

[8] Giulio Franzese, Giulio Corallo, Simone Rossi, Markus Heinonen, Maurizio Filippone, and Pietro Michiardi. Continuous-time functional diffusion processes. In *NeurIPS*, 2023. 3

[9] Daniel Geng, Charles Herrmann, Junhwa Hur, Forrester Cole, Serena Zhang, Tobias Pfaff, Tatiana Lopez-Guevara, Yusuf Aytar, Michael Rubinstein, Chen Sun, Oliver Wang, Andrew Owens, and Deqing Sun. Motion prompting: Controlling video generation with motion trajectories. In *CVPR*, pages 1–12, 2025. 3

[10] Xuehai He, Shuohang Wang, Jianwei Yang, Xiaoxia Wu, Yiping Wang, Kuan Wang, Zheng Zhan, Olatunji Ruwase, Yelong Shen, and Xin Eric Wang. Mojito: Motion trajectory and intensity control for video generation. *arXiv preprint arXiv:2412.08948*, 2024. 3

[11] Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P. Kingma, Ben Poole, Mohammad Norouzi, David J. Fleet, and Tim Salimans. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022. 1, 3

[12] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J. Fleet. Video diffusion models. In *NeurIPS*, pages 8633–8656, 2022. 1, 3

[13] Ruineng Li, Daitao Xing, Huiming Sun, Yuanzhou Ha, Jinglin Shen, and Chiuman Ho. TokenMotion: Decoupled motion control via token disentanglement for human-centric video generation. In *CVPR*, pages 1951–1961, 2025. 3

[14] Jae Hyun Lim, Nikola B. Kovachki, Ricardo Baptista, Christopher Beckham, Kamyar Azizzadenesheli, Jean Kossaifi, Vikram Voleti, Jiaming Song, Karsten Kreis, Jan Kautz, Christopher Pal, Arash Vahdat, and Anima Anandkumar. Score-based diffusion models in function space. *Journal of Machine Learning Research*, pages 1–62, 2025. 3

[15] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *ICLR*, 2023. 5

[16] Isabella Liu, Linghao Chen, Ziyang Fu, Liwen Wu, Haian Jin, Zhong Li, Chin Ming Ryan Wong, Yi Xu, Ravi Ramamoorthi, Zexiang Xu, and Hao Su. OpenIllumination: A multi-illumination dataset for inverse rendering evaluation on real objects. In *NeurIPS*, pages 36951–36962, 2023. 5

[17] Yuan Liu, Cheng Lin, Zijiao Zeng, Xiaoxiao Long, Lingjie Liu, Taku Komura, and Wenping Wang. SyncDreamer: Generating multiview-consistent images from a single-view image. In *ICLR*, 2024. 3

[18] Zhixuan Liu, Peter Schaldenbrand, Yijun Li, Long Mai, Aniruddha Mahapatra, Cusuh Ham, Jean Oh, and Jui-Hsien Wang. TokenDial: Continuous attribute control for text-to-video generation in visual dial space. *arXiv preprint arXiv:2603.27520*, 2026. 3

[19] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. *arXiv preprint arXiv:2310.04378*, 2023. 3

[20] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *ICLR*, 2022. 3

[21] Shuwei Shi, Biao Gong, Xi Chen, Dandan Zheng, Shuai Tan, Zizheng Yang, Yuyuan Li, Jingwen He, Kecheng Zheng, Jingdong Chen, Ming Yang, and Yinqiang Zheng. MotionStone: Decoupled motion intensity modulation with diffusion transformer for image-to-video generation. In *CVPR*, pages 22864–22874, 2025. 3

[22] Xiaoyu Shi, Zhaoyang Huang, Fu-Yun Wang, Weikang Bian, Dasong Li, Yi Zhang, Manyuan Zhang, Ka Chun Cheung, Simon See, Hongwei Qin, Jifeng Dai, and Hongsheng Li. Motion-I2V: Consistent and controllable image-to-video generation with explicit motion modeling. In *ACM SIGGRAPH Conference Papers*, pages 1–11, 2024. 3

[23] Yichun Shi, Peng Wang, Jianglong Ye, Long Mai, Kejie Li, and Xiao Yang. MVDream: Multi-view diffusion for 3d generation. In *ICLR*, 2024. 3

[24] Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, Devi Parikh, Sonal Gupta, and Yaniv Taigman. Make-A-Video: Text-to-video generation without text-video data. *arXiv preprint arXiv:2209.14792*, 2022. 1, 3

[25] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *ICLR*, 2021. 3

[26] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *ICML*, pages 32211–32252, 2023. 3

[27] Team Wan. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. 3, 5

[28] Kaisiyuan Wang, Qianyi Wu, Linsen Song, Zhuoqian Yang, Wayne Wu, Chen Qian, Ran He, Yu Qiao, and Chen Change Loy. MEAD: A large-scale audio-visual dataset for emotional talking-face generation. In *ECCV*, 2020. 5

[29] Shengming Yin, Chenfei Wu, Jian Liang, Jie Shi, Houqiang Li, Gong Ming, and Nan Duan. DragNUWA: Fine-grained control in video generation by integrating text, image, and trajectory. *arXiv preprint arXiv:2308.08089*, 2023. 3

[30] Biao Zhang and Peter Wonka. Functional diffusion. In *CVPR*, pages 4723–4732, 2024. 3

[31] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, pages 586–595, 2018. 6

[32] Zhongwei Zhang, Fuchen Long, Zhaofan Qiu, Yingwei Pan, Wu Liu, Ting Yao, and Tao Mei. MotionPro: A precise motion controller for image-to-video generation. In *CVPR*, pages 27957–27967, 2025. 3